Moving Aeza's catalog to Shopify

How we connect brands' Shopify stores to our existing products without redoing the AI work

For team discussion. This explains, in plain language and with real examples, how we plan to switch from CSV uploads to products fetched directly from each brand's Shopify store, while **keeping all the AI work already done** (descriptions, colours, embeddings, ghost-mannequin images) and **without changing the existing `products` table**.

Item	Detail
Status	Plan / proposal. Nothing in the main database has been changed.
Based on	Read-only analysis of the main `products` table (4,749 products, 27 brands) and the live Shopify sync of the first connected brand, Sands of India (224 products)
Decisions needed	Section 10

Contents

1. The problem in one paragraph
2. What we found in the data
3. The idea, explained simply
4. Left, right and the new table: match and merge
5. Which data comes from where
6. Four real examples
7. What happens when a brand installs
8. Is it automatic? (yes)
9. Saving AI cost
10. Rollout plan and decisions needed

1. The problem in one paragraph

Until now every brand sent us a CSV. We imported it into the `products` table and ran expensive automations on every product **once**: AI descriptions, colour extraction, attribute tagging (fabric, fit, occasion...), text and image embeddings, and ghost-mannequin images for try-on. Now brands install our private Shopify app and we get their products directly, always up to date. The question: **how do we use Shopify as the source of truth without paying again for AI work we already have, and without breaking anything that uses the current products?**

2. What we found in the data

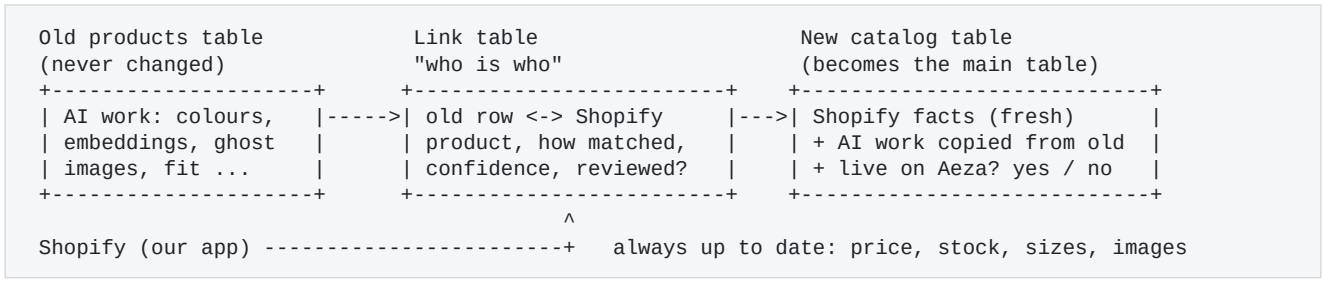
Read-only checks on the real data (Sands of India compared product by product; all brands checked for patterns):

Finding	Numbers	What it means
One row in `products` = one Shopify product	Sands: 221 rows vs 224 Shopify products	The two line up naturally
The product's web name (`handle`) matches exactly	**221 of 221** matched one-to-one	Handle is our main matching key
Size codes (SKUs) confirm the match	220 of 221 confirmed	A strong double-check
SKUs are NOT always unique	Sands uses the same SKUs on two different '2816' tops	Never match on SKU alone
Some products are new in Shopify	3 Sands products have no old row	Only these need the paid AI pipeline
Old prices are out of date	117 of 221 Sands prices changed	Shopify must own price and stock
The AI work exists and is valuable	Sands: 221 analysed, 221 text embeddings, 201 image embeddings, 184 ghost images	Must be kept, not regenerated
Many other tables point at each product's ID	~15 tables: ghost images, user interactions, size charts, creator picks, trending...	Product IDs must never change
Some brands saved one row per colour	e.g. Keshubaba `BD-412` (Blue and Orange as separate rows)	We must match per colour for those brands
Some brands were imported twice	Manaca: 274 duplicate groups with made-up `TEMP-SKU-...` codes	Duplicates need a 'keep one' rule
Older imports have no size details	1,723 rows have no variants data	Shopify fills this in once connected

3. The idea, explained simply

Think of a school:

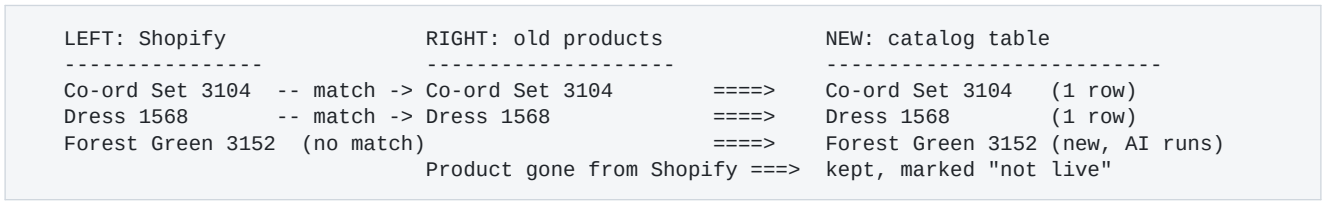
- **The old notebook** (`products` table): a teacher spent months writing it. Every student has a photo, a report card and notes. It was expensive to make, but some details (like phone numbers) are out of date.
- **The school office list** (Shopify): always correct about who is enrolled, their class and fees, but no photos or report cards.
- **What we do**: we do **not** scribble in the old notebook. We start a **new register**. For each student on the office list we find them in the old notebook, copy their photo and report card, and take the up-to-date details from the office list. A new student gets their photo taken (that's the paid AI work). Students who left are marked 'left'.



Key trick: a matched product keeps the same ID in the new table as in the old one. Ghost images, user history, size charts and creator picks all point at that ID, so they keep working without moving anything.

4. Left, right and the new table: match and merge

It is **not** a plain union (that would list every product twice). It is **match, then merge**: pair each Shopify product with its old row, then build **one** row from the pair.



Case	Left (Shopify)	Right (old)	New table gets
Matched pair (most products)	yes	yes	1 merged row, keeps the old ID
New in Shopify	yes	no	1 new row; AI pipeline runs once
Only in old table (removed from store, or brand not installed yet)	no	yes	1 row copied as-is, marked 'not live' or 'from CSV'
Duplicates (e.g. imported twice)	yes	yes, twice	1 row (the best copy); the other recorded as duplicate

5. Which data comes from where

Every column has one owner. That is what keeps the AI work safe and the shop data fresh.

Owner	Columns	When it changes
Shopify (refreshed on every change)	Title, description, web name (handle), images, main image, price, min/max price, MRP, stock, sizes, size SKUs, active/draft, deleted	Updated automatically. **No AI cost.**
Aeza AI (copied from the old table, never overwritten by Shopify)	AI description, category, gender, colours (primary, family, hex, base), fabric, fit, neckline, lengths, rise, occasion, season, aesthetic, activity, luxury level, pairing advice, layering, breathability, body-part tag, text and image embeddings, ghost images	Re-run **only if its inputs change** (section 9)
Aeza (kept)	Brand link, collection, internal tags	Unchanged
The brand's choice	Live on Aeza? (picked in our Shopify app: 'Show all' or 'Only selected')	When the brand changes it

6. Four real examples

Example 1: a perfect match (Sands of India, Co-ord Set 3104), like 220 of their 221 products

	Old row	Shopify	New catalog row
ID	`017a41d4...`	product `10025366651160`	`017a41d4...` (same ID kept)
Web name	`soivocs76-black-poplin...`	`soivocs76-black-poplin...` (same)	same
Price	Rs. 2,449 (old)	Rs. 2,353 (today)	**Rs. 2,353** (from Shopify)
Sizes and stock	from the CSV, outdated	XS 13, S 23, ...	**from Shopify**
Colours, fabric, fit, occasion, AI description, embeddings, ghost images	already done	-	**copied: Rs. 0 spent**
Live on Aeza?	-	brand ticked it	**yes**

Example 2: a brand-new product (Sands, Forest Green 3152)

In Shopify, no old row. A new catalog row is created; the AI pipeline runs **once, for this product only**.

Example 3: one product saved as one row per colour (Keshubaba `BD-412`; likely also Tanoti, Soulinnate)

Old table: 2 rows (Blue, Orange), each with its own ghost images and colours. In Shopify it is probably **one product with a Colour option** (to be confirmed when they install). The new table keeps **one row per product per colour**. Each keeps its old ID and AI work, and the sizes are matched to the right colour using their SKUs.

Example 4: the same product imported twice (Manaca, 274 cases)

Two old rows with made-up `TEMP-SKU-...` codes, one Shopify product. We keep the copy with the most AI work and history and record the other as 'duplicate, not used'. Nothing is deleted.

> Why we never match on SKU alone: Sands of India's Pink and Peach '2816' tops share the same size codes (`SISH0061024XS`, `...0S`, `...0M`, `...0L`). Matching by SKU would attach the Peach top's AI data to the Pink one. The web name (handle) is unique in Shopify, so it is the safe key; SKUs are only a double-check.

7. What happens when a brand installs

Two stages

Stage	When	What happens
Stage 1: once, for everyone	Day 1 of the switch	Create the new table and copy all ~4,750 products into it with their AI work and the same IDs. Every brand is labelled **'from CSV, not connected yet'** and stays live exactly as today. Shoppers see no change.
Stage 2: brand by brand	Whenever each brand installs	Matching runs for that brand only. Their rows get connected to Shopify and switch to **'connected'**. From then on Shopify keeps them up to date.

```
Day 1:      New table = all ~4,750 products (copied)   ->  every brand "from CSV"
Sands:      already installed -> connect now            ->  "connected"
Rawayi:     gets link, installs -> matching runs        ->  "connected"
Brand X:    gets link next week -> matching runs        ->  "connected"
...
Eventually: every brand "connected"; the old products table is only an archive
```

We cannot connect a brand before it installs: until then we have no access to its Shopify data.

What the brand sees (same for every brand)

Step	Brand sees	Behind the scenes
1. Open the link -> **Install**	Shopify permission page	Our app gets access to their store (saved encrypted)
2. App opens	Intro (AI stylist, try-on, sizing) -> **Onboard your brand**	Shopify starts exporting the whole catalogue
3. Short form	Brand name, website, category, email, phone -> **Onboard**	All products and sizes arrive in about a minute
4. Dashboard	'You're connected to Aeza', their products, toggle **Show all on Aeza / Only selected**	**Matching runs** for this brand
5. Pick products -> **Save**	'Your products are on their way to Aeza'	Chosen products become live in the new table

Existing brand (e.g. Rawayi) vs a completely new brand

	Existing brand (already in old table)	New brand (nothing in old table)
What they see	Exactly the same screens	Exactly the same screens
Matched products	Linked: keep AI work, fresh price/stock	None
New products	Only those added since their CSV	All of them
AI cost	Only new products	Only products they make live (section 9)

After onboarding, automatically

Brand does in Shopify	Aeza catalog	AI cost
Changes a price or stock	Updated within about a minute	Rs. 0
Sells out a size	That size shows 0	Rs. 0
Adds a product	Appears in their dashboard; live if 'Show all', otherwise when ticked	Once, when it goes live
Changes photos	Ghost images and colours redone for that product	That product only
Deletes or archives a product	Disappears from Aeza (kept for history)	Rs. 0
Uninstalls	All their products disappear from Aeza	Rs. 0
A customer buys on Aeza	Order appears in their Shopify as Paid, tagged 'Aeza', stock reduced	-

8. Is it automatic? Yes

Nobody maps products by hand. On every install the code:

1. Fetches the brand's whole Shopify catalogue (already working today).
2. Finds each product's old row by **same brand + same web name (handle)**.
3. Double-checks with the **size SKUs**; for colour-split brands, matches each colour by its SKUs.

4. **Sure matches**: links them, refreshes price/stock/sizes, keeps the AI work.
5. **New products**: adds them; the AI pipeline starts when they go live.
6. **Products gone from Shopify**: marks them not live.
7. Keeps everything in sync afterwards through Shopify's change notifications (webhooks).

The only manual part: a short review list

When the code isn't sure, it **does not guess**; it puts the case on a review list. Each one is a single decision ('same product' or 'treat as new'). Until reviewed, the product just keeps its current data, so nothing breaks.

Goes to review when...	Example
One Shopify product matches two old rows	Manaca's double imports
The web name matches but the SKUs point at a different product	Sands' shared-SKU '2816' tops
No web-name match, only a weak title match	Products renamed in Shopify after the CSV

Brand type	Automatic	Needs review
Clean brands like **Sands of India**	220 of 221	1
Colour-split brands (Keshubaba, Tanoti, ...)	Most	A few, until their Shopify colour setup is confirmed
Messy imports (**Manaca**: 274 duplicate groups)	The clear ones	Likely the most; check before sending their link

9. Saving AI cost

Change	What re-runs	Cost
Price, MRP, stock, sizes added or removed	Nothing	Rs. 0 (most of all changes)
Title or description	Search text and the cheap text embedding (AI description optional)	Tiny
Photos (compared by image address, ignoring Shopify's version number)	Ghost images, colour extraction, image embeddings	That product only
New product	Full pipeline	Once
Deleted, archived or unpublished	Nothing (hidden, kept for history)	Rs. 0

- **Sands of India today:** 221 products reuse everything, 117 get free price fixes, only **3 products** need the paid pipeline.
- **For new brands:** run the AI pipeline **only for products the brand makes live**, not the whole catalogue. A brand with 500 products that picks 80 costs 80 AI runs, not 500.

10. Rollout plan and decisions needed

Rollout (each step is safe and reversible)

1. **Match report for all 27 brands** (read only, changes nothing): per brand, how many products would link automatically, need review, are colour-split or duplicates. Tells us which brands are ready for a link.
2. **Create the new tables** next to `products` (new catalog table, sizes table, link table) on a Neon test branch first. The `products` table itself is never changed.
3. **Stage 1 copy**: fill the new table with all ~4,750 products and their AI work (same IDs).

4. **Compare old and new side by side**; fix anything that differs.
5. **Connect Sands of India** (already installed), then switch the app and AI pipelines to read the new table.
6. **Send links brand by brand**; each install connects itself automatically.
7. **Safety net**: if anything looks wrong, point the app back at `products`. It never changed, so going back is instant.

Questions for the team

#	Question	Why it matters
1	Which parts of the app read the `products` table? (search, try-on, cart, size charts, recommendations, creator storefronts)	They switch to the new table
2	How are the AI pipelines started today? (a status set to 'pending', or a jobs table?)	New products must trigger them automatically
3	Do we run AI only for products a brand makes live (recommended) or for every product?	Cost
4	Colour-split brands: one row per product, or one per product + colour (recommended, matches today's data)?	Ghost images and colours are per colour
5	Who handles the review list (product or ops team)?	A few clicks per brand
6	The old `shopify_installations` table stores access tokens: clear it once brands move to the new apps?	Security

> **Summary:** keep the old `products` table untouched, build a new catalog table that combines **Shopify's fresh facts** with the **AI work we already paid for**, keep every product's ID, connect brands one by one as they install (automatically), and only pay for AI on genuinely new or changed products.